

Razvoj web aplikacija - Predavanje 12

Production, deploy i završni polish

mag.ing. Josip Torić (SIT UNIZD)

11.01.2027.



Studij informacijske tehnologije
Sveučilišta u Zadru

Plan predavanja

- 1 Razlika dev vs production okruženje
- 2 Environment varijable i konfiguracija
- 3 CORS: što je i zašto treba
- 4 Deploy backenda na Railway
- 5 Deploy frontenda (statički build)
- 6 CI/CD pipeline
- 7 Priprema za 2. obranu
- 8 Commitovi za ovo predavanje

Na kraju predavanja: aplikacija je dostupna na javnom URL-u, s CI pipelineom koji provjerava kod.

Nakon ovog predavanja student može:

- objasniti razliku između dev i production konfiguracije
- konfigurirati CORS na backendu za komunikaciju s frontendom
- deployati FastAPI backend na Railway ili Render
- konfigurirati Vite build za produkciju s environment varijablama
- postaviti osnovni CI pipeline za lint, testove i build

Dev vs Production

	Development	Production
Backend	uvicorn --reload, local-host:8000	Gunicorn + Uvicorn workers, javni URL
Frontend	Vite dev server, localhost:5173	Statički build (dist/)
Baza	Docker lokalno	Managed PostgreSQL
Debug	Swagger UI, detalji grešaka	Bez Swaggera, generičke greške
Env	.env file	Platform env vars

Nikad ne deployajte s `--reload` ili s `DEBUG=True`. Production konfiguracija skriva detalje grešaka od korisnika.

Production mindset

Razvoj i produkcija zahtijevaju različit način razmišljanja:

	Development	Production
Greške	Detaljan stack trace	Generička poruka korisniku, detalji u logovima
Padanje	Prihvatljivo - restart	Mora se oporaviti automatski
Podaci	Seed podaci, testni	Pravi korisnici, pravi podaci
Sigurnost	Relaksirano	Sve zaključano

Princip: **fail gracefully**. Umjesto bijelog ekrana, korisnik vidi „Nešto je pošlo po krivu, pokušajte kasnije”. Detalji greške idu u logove, ne korisniku.

Security checklist za deploy

- 1 **HTTPS svugdje** - nikad HTTP u produkciji (Railway/Render to rade automatski)
- 2 **Env vars za tajne** - JWT_SECRET, DATABASE_URL nikad u kodu
- 3 **Jaki JWT_SECRET** - minimalno 32 znaka, random generirani
- 4 **CORS ograničen** - samo vaš frontend origin, ne *
- 5 **Swagger isključen** - docs_url=None u produkciji
- 6 **Dependency audit** - pip audit i npm audit za poznate ranjivosti

12-Factor App: relevantni principi

Princip	Što znači za nas
Config u envu	Sve konfigurabilno kroz env varijable, ne hardkodirano
Stateless procesi	Server ne čuva stanje između requestova (session je u tokenu)
Port binding	Aplikacija sluša na portu koji joj je zadan (\$PORT)
Dev/prod paritet	Lokalni dev je što sličniji produkciji
Logovi kao stream	print/logging na stdout, platforma ih skuplja

Ovi principi nisu akademski - Railway i Render ih očekuju. Aplikacija koja ih slijedi deploja se bez problema, aplikacija koja ih ne slijedi zahtijeva workarounde.

Environment varijable: backend

```
# app/core/config.py
from pydantic_settings import (
    BaseSettings, SettingsConfigDict
)

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env")

    DATABASE_URL: str
    JWT_SECRET: str
    CORS_ORIGINS: str = "http://localhost:5173"
    DEBUG: bool = False

settings = Settings()
```

Pydantic BaseSettings automatski čita iz env varijabli. U developmentu koristi .env file, u produkciji čita iz platform env vars.

.env nikad ne smije biti committan u git - stavite ga u .gitignore. U repozitoriju držite .env.example s praznim vrijednostima.

Environment varijable: frontend (Vite)

```
# .env.development
VITE_API_URL=http://localhost:8000

# .env.production
VITE_API_URL=https://moj-api.railway.app
```

```
// src/api/client.ts
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
})
```

Vite automatski učitava `.env.development` u dev modu i `.env.production` za build. Sve varijable moraju počinjati s `VITE_` da budu dostupne u kodu.

Ove varijable se ugrađuju u build (`npm run build`) - nisu runtime, nego build-time.

CORS: Cross-Origin Resource Sharing

Browser blokira requestove s jedne domene na drugu (Same-Origin Policy). CORS je mehanizam koji dopušta iznimke.

Scenario	Origin	Blocked?
Dev frontend → dev backend	:5173 → :8000	Da (različiti portovi)
Prod frontend → prod backend	app.com → api.app.com	Da (različita subdomena)
Postman → backend	nema origin	Ne (nije browser)

Backend mora eksplicitno dopustiti origine s kojih prima requestove. Bez CORS konfiguracije, frontend neće moći komunicirati s API-jem iz browsera.

CORS konfiguracija u FastAPI

```
# app/main.py
from fastapi.middleware.cors import CORSMiddleware
from app.core.config import settings

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS.split(","),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Parametar	Značenje
allow_origins	Koji origini smiju slati requestove
allow_credentials	Smiju li se slati cookieji
allow_methods	Koje HTTP metode (GET, POST...)
allow_headers	Koji headeri (Authorization...)

Nikad ne koristite `allow_origins=["*"]` u produkciji - to dopušta bilo kojoj stranici da šalje requestove na vaš API.

Preflight request: kad i zašto

Browser šalje **OPTIONS** request prije stvarnog requesta u određenim situacijama:

	Simple request	Preflighted request
Metode	GET, POST, HEAD	PUT, PATCH, DELETE
Headeri	Samo standardni	Custom headeri (Authorization)
Content-Type	form-data, text/plain	application/json
Preflight	Ne	Da - OPTIONS prvo

Naši API pozivi šalju Authorization header i Content-Type: application/json - svaki request je preflighted. Browser prvo pita „smijem li?” (OPTIONS), backend odgovori s CORS headerima, tek onda ide stvarni request.

Content Security Policy

CSP je HTTP header koji kaže browseru odakle smije učitavati resurse:

- **Bez CSP-a:** maliciozna skripta može učitati JavaScript s bilo kojeg URL-a
- **S CSP-om:** browser odbija sve resurse koji nisu s odobrenih origina

Direktiva	Kontrolira
default-src 'self'	Sve resurse s istog origina
script-src 'self'	JavaScript samo s našeg servera
connect-src	API pozivi - ovdje dodajemo backend URL
img-src	Odakle smiju dolaziti slike

CSP je dodatni sloj zaštite od XSS-a. Čak i ako napadač ubaci skriptu, browser je neće izvršiti ako origin nije odobren.

Deploy opcije

Platforma	Backend	Frontend	Baza
Railway	Python buildpack	Static site	Railway PostgreSQL
Render	Python, Docker	Static site	Render PostgreSQL
Fly.io	Docker	CDN	Fly PostgreSQL

Railway i Render imaju besplatne tierove dovoljne za studentski projekt. Odaberite jednu platformu i držite se nje.

Deploy backenda na Railway

```
# 1. Dodajte Procfile u api/  
# Procfile  
web: uvicorn app.main:app --host 0.0.0.0 --port $PORT  
  
# 2. requirements.txt: rucno održavana lista  
# direktnih ovisnosti (fastapi, uvicorn,  
# sqlalchemy, asyncpg, alembic, pyjwt, bcrypt...)  
# Ne generirajte ga s "uv pip freeze" - to zaledi  
# i tranzitivne ovisnosti i tesko se održava.  
  
# 3. Na Railway:  
# - New Project > Deploy from GitHub repo  
# - Root directory: api  
# - Add PostgreSQL plugin  
# - Set env vars: DATABASE_URL, JWT_SECRET, CORS_ORIGINS
```

Railway automatski detektira Python projekt, instalira dependencije i pokreće Procfile naredbu. \$PORT je env varijabla koju Railway postavlja

Deploy frontenda

```
# 1. Build za produkciju
cd web
npm run build # generira dist/ folder

# 2. dist/ sadrzi staticke fileove:
# index.html, assets/main-abc123.js, assets/style-def456.css

# 3. Na Railway/Render:
# - New Static Site > Deploy from GitHub
# - Build command: npm run build
# - Publish directory: dist
# - Set VITE_API_URL env var
```

SPA routing zahtijeva konfiguraciju: sve rute moraju servirati `index.html`. Na Railway/Render postavite **fallback** na `index.html` za 404 stranice.

CDN i caching strategija

Statički asseti (JS, CSS, slike) se serviraju s CDN-a za brži pristup:

- Vite generira hashirane nazive: `main-abc123.js`
- Hash se mijenja samo kad se kod promijeni
- Server može cachirati hashirane fileove zauvijek (`Cache-Control: immutable`)
- Novi deploy = novi hash = browser automatski dohvati novu verziju

`index.html` se **nikad ne cachira** (ili s kratkim `max-age`) jer sadrži linkove na hashirane asete. Kad se deploy promijeni, novi `index.html` referencira nove fileove.

Build optimizacije

`npm run build` generira optimizirani bundle za produkciju:

Optimizacija	Što radi
Tree shaking	Uklanja nekorišteni kod iz bundlea
Code splitting	Lazy-loaded rute postaju zasebni chunkovi
Minification	Skraćuje nazive varijabli, uklanja whitespace
CSS extraction	CSS u zasebnom fileu (paralelno učitavanje)

Rezultat: umjesto jednog velikog JS filea, dobijemo glavni chunk + chunk po ruti.
Korisnik preuzima samo kod za stranicu koju gleda.

Performance budžet

Koliko JavaScripta je previše?

Bundle veličina	Procjena
< 100 KB (gzipped)	Odlično
100-250 KB	Prihvatljivo za SPA
250-500 KB	Treba optimizirati
> 500 KB	Problem - korisnici na sporom internetu čekaju

Alati za analizu: `npx vite-bundle-visualizer` pokazuje koji paketi zauzimaju koliko prostora. Često su krivci: `moment.js`, `lodash` (cijeli), UI biblioteke. `Vue` + `Vue Router` + `Pinia` + `Axios` zajedno su oko 50 KB gzipped - razumno.

SPA fallback: zašto je potreban

Kad korisnik direktno upiše `app.com/clubs/5/lifters`:

- 1 Browser šalje GET `/clubs/5/lifters` web serveru
- 2 Web server traži file `clubs/5/lifters/index.html` - ne postoji
- 3 Bez fallbacka: **404 Not Found**
- 4 S fallbackom: servira `index.html` → Vue Router preuzima → renderira `LiftersView`

Ovo je razlika između MPA (server zna rute) i SPA (klijent zna rute). Server mora sve nepoznate rute preusmjeriti na `index.html`.

Migracije na produkcijskoj bazi

```
# Na Railway: dodajte u start komandu  
# Procfile  
release: cd api && alembic upgrade head  
web: uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

release komanda se izvršava **prije** pokretanja aplikacije, pri svakom deployu. Alembic automatski primijeni sve pending migracije.

Nikad ručno ne mijenjajte production bazu. Sve promjene idu kroz migracije:

- 1 Lokalno: `alembic revision --autogenerate -m "opis"`
- 2 Testirajte lokalno: `alembic upgrade head`
- 3 Commitajte migraciju
- 4 Deploy - release komanda automatski pokrene migraciju

CI pipeline: GitHub Actions

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - uses: astral-sh/setup-uv@v5
      - run: uv pip install --system -r api/requirements.txt
      - run: cd api && ruff check . && ruff format --check .
      - run: cd api && pytest

  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
      - run: cd web && npm ci
      - run: cd web && npm run lint
      - run: cd web && npm run type-check
      - run: cd web && npm run test -- --run
      - run: cd web && npm run build
```

CI/CD: što i zašto

Korak	Što provjerava
Lint (ruff/eslint)	Formatiranje, stilske greške, potencijalni bugovi
Testovi	pytest i Vitest iz Predavanja 11 - logika radi, nema regresija
Build (npm run build)	Frontend se kompajlira bez grešaka

Pipeline se pokreće na svaki push i PR. Ako bilo koji korak padne, commit je označen crvenim X-om. Ovako broken kod ne dolazi u produkciju.

Za ovaj kolegij CI nije obavezan, ali je bonus. Ako ga postavite, pokazuje zrelost projekta na obrani.

CD: Continuous Deployment

CI provjerava kvalitetu koda. CD automatski deploja kod koji prođe provjere:

- 1 Push na `main` branch
- 2 CI pokrene lint + testove + build
- 3 Ako sve prođe → automatski deploy na produkciju
- 4 Ako padne → deploy se ne dogodi, developer dobije obavijest

Railway i Render automatski deployaju svaki push na `main` - to je CD bez ikakve konfiguracije. Za složenije sustave koristimo staging environment (test server) prije produkcije.

Rollback strategija: ako novi deploy pokvari aplikaciju, revertamo commit i pushamo - automatski se deploja prethodna verzija.

Monitoring u produkciji

Aplikacija u produkciji mora biti nadgledana:

Što	Alat	Zašto
Error tracking	Sentry	Hvata exceptions i šalje alert
Uptime monitoring	UptimeRobot	Provjera je li server živ (health check)
Logovi	Railway logs	Debug problema u produkciji
Performance	Lighthouse	Brzina učitavanja frontenda

Bez monitoringa ne znate da aplikacija ne radi dok vam se student ne javi. Health check endpoint (GET /health) je minimum - monitoring alat ga pinguje svake minute.

Logging strategija

- **Strukturirani logovi:** JSON format s timestamp, level, message, context
- **Log levels:** DEBUG (dev), INFO (normalne operacije), WARNING (sumnjivo), ERROR (greška)
- **Što logirati:** login pokušaji, CRUD akcije, greške, spori upiti
- **Što ne logirati:** lozinke, tokene, osobne podatke (GDPR)

U produkciji logovi idu na stdout - platforma (Railway) ih skuplja i prikazuje u dashboardu. Ne pišite u fileove na serveru - kontejneri su efemerni.

Checklist prije deploya

- 1 `.env` je u `.gitignore` (nikad committan)
- 2 `.env.example` postoji s praznim vrijednostima
- 3 CORS origins postavljeni na frontend URL (ne *)
- 4 Swagger UI isključen u produkciji (`docs_url=None` kad `DEBUG=False`)
- 5 `JWT_SECRET` je dugačak random string (ne `secret123`)
- 6 Frontend build radi: `npm run build` bez grešaka
- 7 `VITE_API_URL` postavljen na production backend URL
- 8 Migracije se automatski pokreću pri deployu
- 9 Health check endpoint radi: `GET /health`

Database backup i recovery

Podaci u produkciji su najvredniji dio aplikacije:

- **Automatizirani backupi:** Railway/Render rade dnevne backupe managed PostgreSQL-a
- **Point-in-time recovery:** mogućnost vraćanja baze na bilo koji trenutak u zadnjih X dana
- **Testiranje restora:** backup koji nikad niste testirali nije backup

Scenarij: pogrešna migracija obriše stupac s podacima. Bez backupa - izgubljeni podaci. S backupom - restore na stanje prije migracije. Zato nikad ne radite DROP COLUMN bez prethodne provjere i backupa.

Što dalje nakon kolegija

Tehnologije koje nadograđuju ono što ste naučili:

Tehnologija	Zašto
Nuxt	Server-side rendering za Vue - SEO, brži first paint
Docker	Kontejnerizacija: isti environment lokalno i na serveru
WebSockets	Real-time komunikacija bez pollinga (chat, live score)
PWA	Progressive Web App - offline support, instalacija na mobitel
GraphQL / tRPC	Type-safe API komunikacija bez pisanja duplog koda
TanStack Query	Server-state s cachingom i retryjem - zamjenjuje ručni loading/error kod iz P9

TypeScript i testiranje (Vitest, Playwright) su dio ovog kolegija. Ono što ste naučili je solidan temelj - svaka od ovih tehnologija nadograđuje iste principe.

Commitovi za Predavanje 12

- chore: configure production env and CORS settings for deployment
- feat(web): add build-time env config and runtime API base handling
- ci: add pipeline for lint/test/build (api + web)
- docs: add deploy guide (Railway) and production checklist
- refactor: final cleanup, remove dead code, tighten types

Na satu: deployamo aplikaciju na Railway, gledamo logove, testiramo javni URL.

Priprema za 2. obranu (18.01.)

- 1 Aplikacija je deployana i dostupna na javnom URL-u
- 2 Demonstrate: login, CRUD, pravila, error stanja
- 3 Pokažite Network tab u DevToolsima - requestovi i responsovi
- 4 Budite spremni objasniti:
 - Kako frontend hendla 401? Gdje je interceptor?
 - Što se dogodi kad korisnik ručno upiše admin URL?
 - Gdje držite stanje? Zašto store, a ne lokalni ref?
 - Kako ste konfigurirali CORS i zašto?
 - Kako radi deploy pipeline?
- 5 Mini promjena uživo: dodaj polje u formu, prikaži novi podatak

Rekapitulacija: cijeli stack

Backend (P1-P6)

- 1 Skeleton, config, health
- 2 Modeli, ORM, migracije
- 3 Autentikacija (JWT)
- 4 Autorizacija (role, ownership)
- 5 CRUD, validacija, paginacija
- 6 Poslovna pravila, workflow

Frontend (P7-P12)

- 7 SPA, router, layout
- 8 API komunikacija, auth flow
- 9 State management, forme
- 10 Admin tablice, filteri, UI
- 11 Testiranje: Vitest, Playwright
- 12 Deploy, CORS, CI/CD

12 predavanja, jedan cjeloviti full-stack projekt. Svako predavanje dodaje novi sloj na postojeći rad.

Završne napomene

- Projekt je **vaš** - tema je vaš izbor, ali zahtjevi su jasni
- Koristite AI za učenje, ne za izbjegavanje razmišljanja
- Kod koji ne razumijete je kod koji ne možete obraniti
- Pitajte kad zapnete - na predavanjima ili mailom

Pitanja?